
C2 Naval Appchain

Reference implementation accompanying the Master's thesis **"Decentralized Communication Architecture for C2 Systems Using Blockchain Technology"** (CINAV / Portuguese Naval Academy).

The thesis models a naval Task Group's command-and-control system as a two-layer blockchain. The Task Group's ships form a permissioned, Byzantine-Fault-Tolerant Layer 1 backbone. Drones operate on a Layer 2 validity rollup, where STRK tokens represent operational authorization to carry out a mission. This repository contains the actual stack used to build, run, and evaluate that architecture.

What this is

- A Hyperledger Besu QBFT network (Layer 1) — the Task Group backbone.
- A Madara Starknet appchain (Layer 2) — the drone-swarm rollup.
- A Karnot orchestrator driving an offline proving pipeline: Madara → Cairo PIE → Stwo Circle STARK proof → settlement on the Besu L1.
- A custom Node.js gateway (`Layer2/stwo-gateway/`) implementing the SHARP-compatible HTTP API in front of a locally-built Stwo prover binary, so the entire pipeline runs with no external network calls — a core requirement for the naval use case.
- A Grafana/Prometheus/OpenTelemetry observability stack for block production, mempool, and proving metrics.

See [docs/ARCHITECTURE.md](#) for the conceptual model — what's implemented today versus what's described as future work — and the argument for why a rotating, untrusted L2 sequencer is safe under this design.

Repository layout

<code>compose.yaml</code>	Main stack definition
<code>compose.dynamic.yaml</code>	Optional additional full node (madara-vessel-frigate)
<code>compose.bootstrap-override.yaml</code>	Bootstrap-time overrides
<code>up.sh</code>	Orchestration script (wipes state + brings up the stack)
<code>configs/</code>	Madara, bootstrapper, OTEL, Prometheus configs + TLS fixtures
<code>Layer1/besu/</code>	Besu QBFT genesis, validator keys, config
<code>Layer1/scripts/</code>	L1-side helper scripts (add_validator.sh)
<code>Layer2/contracts/</code>	Cairo contracts (balance_contract)
<code>Layer2/scripts/</code>	L2-side helper scripts (add_fullnode.sh)
<code>Layer2/stwo-gateway/</code>	Custom SHARP-compatible Node.js gateway
<code>helper/</code>	Solidity mock verifier
<code>grafana/</code>	Dashboards and provisioning
<code>docs/</code>	Architecture notes, known issues, build instructions

Quick start

1. Build the custom images from source — see [docs/BUILD.md](#) . This covers the Madara sequencer, bootstrapper-v2, orchestrator, the stwo-gateway, and the standalone Stwo prover binary.
2. Copy `.env.example` to `.env` and fill in your own values (deployer private key, etc.) — **never commit** `.env` .
3. Bring up the stack:

```
./up.sh
```

4. Deploy a drone account and the example contract with `sncast` — see the command sequence in [docs/BUILD.md](#) or the inline comments in `Layer2/contracts/balance_contract/` .

5. Open Grafana at `:3001` to watch block production and proving metrics live.

What's not in this repository

A few things are intentionally excluded — see `.gitignore`:

- `.env`, `orchestrator.env`, and two helper scripts (`helper/deploy_verifier.sh`, `layer1/scripts/fund_l2_accounts.sh`) that contain a hardcoded development private key. These are kept in a separate, non-public location.
- `data/` and `output/` — live chain state and bootstrapper-generated addresses, both fully regenerated on every fresh `./up.sh` run.
- `prover/proving-utils/` — the cloned Stwo prover source and its compiled binary. Built separately per `docs/BUILD.md`, not vendored here.

Status

This is active thesis work with a hard deadline of **July 3, 2026**. Expect the architecture and contracts to evolve. See [docs/KNOWN_ISSUES.md](#) for the current set of open technical problems and the workarounds in place.